

Prototype for Accessible Sortable Table

GSoC Project proposal for KDS

Synopsis

As a part of this project, I would like to develop a new KDS component, a sortable data table. While many pre-existing component libraries provide very flexible and customizable data tables and grids, their accessibility is very limited. With this project, I aim to balance accessibility and functional requirements to create a component that could help empower all, lay the foundation for further work, and, eventually, provide an impactful update for all learners and teachers using Learning Equality products.

Benefits to the Community

This project aims to add a sortable table-cum-data grid component to KDS that could be used by all the products in the Kolibri ecosystem to display large amounts of data in an easy-to-grasp, clear, concise, and accessible manner. The community could then adapt and build upon the table component to create more fine-grained and narrow-scoped wrappers and components to better cater to their specific needs.

A well-thought-out and flexible prototype would make all their future explorations easy and help to reduce a lot of developer overhead. Since the accessibility of tables is not a well-known topic and requires a considerable amount of research on the part of developers, adding this component to the library would ensure that all the developers can enjoy the benefits of accessibility and internationalization in their sites without even being concerned about the same.

Current Status of the Project

The project has a **coreTable** component that uses slots to render tabular layouts. As per my understanding of the implementation details, I reckon that the same is intended to be used as a wrapper component for tabular “layouts” rather than data grids (which are made to work principally with data). Thus, for the prototype, I would like to create a new component from scratch that can cater to our needs. There is also a **SortableHeader** component in KDP, which can act as an excellent reference from both the accessibility and implementation points of view. Support for internationalization (by receiving translations from Kolibri) is present and documented, but not much precedent applies to tabular components.

A series of issues in KDS (#328-#332) have been opened, and they briefly discuss various aspects of designing such a sortable grid. No code contribution has been made to address them.

Goals

The prototype component(s) should be designed in such a way that they can handle the following use cases:

- Static data (Data that is only present for display purposes and does not require to be sorted or filtered)
- Dynamic data (Data that must be sorted according to the user's needs)

The types of tabular data that the project aims to support are as follows:

- Data with horizontal headers
- Data with vertical headers
- Data with biaxial headers (both horizontal and vertical headers)

The accessibility to screen readers of the above component would be of the utmost importance, followed by internationalization. The component should adequately communicate its sorting capabilities to the screen reader. In addition, there should be a way for the users to edit the data of the tables directly. The component should be accompanied by proper documentation so that the developers can easily integrate the same and comprehensive tests that give the confidence that the existing functionality does not break due to new features.

Deliverables

As per my reading into the accessibility and internationalization requirements for the data grids, I have worked on accumulating the following requirements for the new prototype. These are to serve as the guiding rails during the development process and thus would also be the deliverables:

Table Headers

1. The table header tags must be used correctly for tables with single headers to identify the row or column acting as the header. In such tables with single headers, only the first row/column should be allowed to be the headers. ([Reference](#))
2. The header must have the appropriate “rowheader” or “columnheader” roles. ([Reference for implementation](#))
3. The header tags should be used for both rows and columns in tables with horizontal and vertical headers. The headers must be associated with a “scope” attribute to quantify their direction. The headers need not be the first row/columns in such a table. ([Reference](#))

4. We would only concern ourselves with tables with singular headers in each direction. Tables with [irregular headers](#) are considered out of the prototype's scope.
5. If a particular row/column allows for sorting, then the header text of the column/row should be wrapped in a button component. The header should also contain the “aria-sort” attribute to indicate the direction of sorting ([Reference](#))
 - a. By default, the developers should only be required to specify the names/indexes of the columns/rows that allow sorting via props, and the default functionality of ascending and descending sorting should be implemented.
 - b. Developers should also be able to specify the type of data the header precedes over (string/number). This is important because it can lead to [unexpected sorting results](#) if not handled carefully.
 - i. We should default to string values if no type is specified for the column/row. The same should be mentioned in the developer documentation.
 - ii. If the data type is specified as a number for a row/column, then during comparison, we can use the “+” operator to coerce the values to a numeric form (to provide support for number data as strings)
 - iii. A validator function should be present in the component, which also takes care of the validation of the data types.
 - c. Developers can also provide a custom sorting function that can be used to rearrange the columns/rows. In such a case, the developer should also provide a brief description of the sorting logic that is implemented. This would be attached to the associated accessibility metadata of the table (described below), and the value of the “aria-sort” attribute should be set to custom.
 - d. We should add an [aria-live](#) region to the page when enabling row sorting. An aria-live region following the <table> element automatically announces when the sort state changes for visitors using screen readers.
6. The table headers of all the sortable columns/rows must have an icon, which can be used to demarcate the fact that they are sortable. [Chevron](#) arrows are a good candidate for the default icons.
 - a. When the sorting for a particular column is active, the icon must change depending on whether the same is in ascending or descending order.
 - b. The icons used should differ in the shape of the unsorted icon, not just the color and size. The developers should be able to provide icons for all three states (unsorted, sorted-ascending, sorted-descending) via props. They should be aware of this accessibility requirement via the developer documentation.
 - c. All the icons should be enclosed in a span with “aria-hidden=true.”
 - d. Tooltips must be on the active sorting column/row to indicate the same.
 - e. We support only sorting by one row/column at a time. Multiple column/row sorting is out of the prototype's scope.

Please also refer to the [appendix on sorting UX](#).

7. Column/Row headers should be focusable only if they are sortable.
8. Fixed table headers functionality must be supported in the case of larger tables and grids.

Scope Attribute

1. For tables with single headers, the scope attribute should be applied to all the cells of the table (with a value of either “row” or “column”) to indicate the direction of the heading. This is only [crucial](#) for ambiguous, not distinguishable data and large tables.
2. No scope attribute should be applied to the cells if the table has two header directions.

Keyboard Support

1. If the developer wants, all the grid cells should be selectable and have a distinctive UI (in terms of background color or border) showing when they are in focus.
2. Data cells should be focusable using the standard navigation keys, including directional keys like Home, End, Control + Home, and Control + End. ([Reference](#))
3. Only the visible data cells must be focusable and have a *tabindex* associated with them. This is of utmost importance in the cases of large data tables with an associated max-width or max-height. The cells with the *tabindex* attribute must change when scrolling.

Editing Support

1. For all the rows/columns that are editable, the data item must be rendered along with a distinctive icon that conveys to the user that the same is editable. This icon must be wrapped in a button component.
2. The developer must be able to customize the icon and the button through named slots.
3. When clicking the icon, the concept of the cell should change to an input box populated with the current data value and be automatically focused. The edit icon and button should be hidden. ([Reference](#))
4. The user should be able to exit editing mode by pressing the Enter key from the keyboard or pressing the Save Changes button (with a Save or Tick Icon), which now appears on the right end of the cell.
5. There should also be support for a keyboard shortcut (like Escape), which allows the user to exit the editing mode and revert the data to its original value.
6. When the user hovers over the input box, a tooltip must be displayed to convey how the changes made can be saved. We can use the [KTooltip](#) component from KDS for this.
7. After editing the data, the user should be able to click enter and save the changes in the table. The edit icon should now again be visible to the user.
8. The component should emit an event to indicate the value of a data item has changed.
9. We can also add support for the v-model attribute as an additional goal.

Please also see the [appendix discussion](#) for why I prefer the inline editing approach.

Table Caption and Summary

1. All the tables made through the component must have a caption. The developer should be able to overwrite the caption styles via a prop.
 - a. All major components (like cells, rows, columns, headers, etc.) must have unique class names so the developers can override the styling by writing their own CSS.
 - b. Both approaches are necessary to reduce the dependence on “/deep/” selectors across the codebase, which are not too great for readability.
2. The component should allow (and encourage via the developer documentation) the developer to provide a summary of the caption that would be rendered along with the same. ([Reference](#))
3. An off-screen description of the sort functionality of the sorting buttons should be appended to the caption text. The same could also be made visible if the developer opts into it. [Sort hints](#) of the current sorting state and general functionality are great ways to ensure the same.
4. The summary of the table should also show if the content of the table is editable and should name the rows and columns that support the same.

Internationalization

As Kolibri provides internationalization support with the help of the [\\$tr utility](#) supplied by the [Vue-Intl](#) package, we must allow the developers to provide translations to the component, which would be merged with the translations auto-generated by the element for other non-visible text-like summaries (adding descriptions of columns and rows that are sortable or editable).

General Component Requirements

1. The developers should be able to provide a “styles” prop to the component to override the element's styling.
2. The component should allow the row backgrounds to be striped (vertically or horizontally) so that the content of different rows can be easily distinguished.
3. If the table has a scroll associated with it (that is, when the height/width of the table is greater than the max-height/max-width set), then the table must specify the “rowcount” and the “colcount” to properly communicate the number of rows to the reader. The same must also be added to the table summary so that the accessibility aids can convey the same to the users.
4. The component should place the “role” attribute on all the non-header cells with the value “cell.”

Expected Results

In addition to the primary goal of having prototype element(s) that support all of the requirements highlighted above, there are some equally essential results that I would strive to aim for in the project period:

- Perform regular research about accessibility requirements, internationalization, and editing data grids that could be incorporated into the prototype.
- Implementing the sortable table component will be even more convoluted as the requirements are nontrivial. To ensure future ease of access, both the developer and client-facing component documentation must be updated and in sync with the development. All the accessibility features and the rationale between them must be documented.
- A testing suite capable of testing the markup and some accessible features should be built parallel to the component.

Since the component's API would undergo drastic changes during development, I would use the KDS documentation to show the gradual evolution of the component. Each PR adding new functionality to the components would also feature documentation updates where the reviewers can test out the newly added functionality in the flesh and offer remarks and recommendations.

Stretch Goals

I would also like to work on expanding the functionality of the data grid to allow:

- Allowing the table to accept slots for the cells to be rendered instead of just plain textual data. The cell component would be wrapped by the appropriate accessibility markup automatically.
- Implement filtering functionalities for the data and provide an accessible mechanism to apply the said filters.

Out-of-Scope Goals

These are some of the features I came across that would be highly useful for the component that currently must be addressed as out of scope, given the timeline of the project and the fact that we intend to build a functional prototype. These could be implemented in the future and serve as a basis of inspiration.

- Allow sorting of the data grid based on multiple columns in an accessible manner ([Inspiration](#))
- Allow the editing of the data to be governed in different manners: inputs for text content, checks for boolean content, and dropdowns for list content.
- Supporting tables with [irregular headers](#) and [multi-level headers](#) through this component
- For large data grids, pagination of data and asynchronous fetching of data are a must-have feature. However, I do not plan on integrating them into this prototype implementation as they are more about adding nice-to-have functionalities rather than improving accessibility.

Approach

As per the specifications highlighted in the above sections, I would like to create primarily three different components:

- **HTable**: Table with a horizontal header
- **VTable**: Table with a vertical header
- **HVTable**: Table with both a horizontal and vertical header

The reason behind choosing to create three such components is two-fold:

- It would greatly simplify the semantic markup of the cells (header and non-header), both
- It would simplify the efficient implementation of sorting and editing logic.

For a detailed discussion, please [refer to the appendix](#).

Instead of creating separate components for editable and static tables, I would allow these behaviors to be configured by the meaning of props passed to these components. This, in my opinion, would make for a more flexible prototype and even be easier for the end consumers of the library as they wouldn't be required to import different components for different use cases but would be able to modify the behavior of the existing ones with the help of props.

To prevent reuse and duplication, I would like to take a hit-and-trial-based approach where I would first start implementing one of the components, and then when I find myself copying code from other components, I would abstract the same into a new component to reduce duplication. This would save us from premature optimization and keep the components concise. Creating components like "KEditableCell," "KHeaderCell," and "KStaticCell" might be a good approach that could help with abstraction. I would be open to refactoring the code and the style to ensure that the designed components are reusable and neither too large nor too granular.

Timeline

I aim to make weekly to biweekly PRs for the project so that there is enough time for the mentors to review all the changes and suggest improvements before the next sprint cycle. This would also ensure we incrementally develop the component continuously and do not block progress due to unstable requirements and changes for unpredictable reasons.

Period	Tasks
After proposal submission [April 2 - May 26]	- Research additional accessibility and internationalization requirements for the component - Convert the requirements to a technical requirement document (example) for easier development in the later phases - Research other UI libraries on how they allow the customization of styles and address accessibility concerns - Begin with the initial implementation of the components

Week 1 and 2 [May 29 - June 10]	- Implement the primary HTable component with semantic and accessible markup - Implement the general requirements related to the component features and styling - Add sorting capabilities to the same - Create relevant documentation and a test suite for the same
Week 3 and 4 [June 11 - June 24]	- Add support for selectable cells and a distinctive UI - Add support for editing the contents of the table - Update the documentation and test suite
Week 5 [June 25 - July 1]	- Add support for internationalization into the component - Thoroughly test the component for accessibility - Finalize the implementation and the capabilities of the component
Week 6 - 7 [July 2 - July 15]	- Implement the VTable component by reusing the logic of the HTable component - Finalize and publish the VTable component to KDS
Week 8 - 9 [July 16 - July 29]	- Implement the semantic markup for the HVTable component - Create tests to ensure that the component matches the accessibility requirements - Create documentation and test suite for it
Week 10 [July 30 - 5 August]	- Implement the sorting functionalities for the headers - Add tests and documentation
Week 11 [August 6 - August 12]	- Add editing support to the component - Add internationalization
Week 12 [August 13 - August 19]	- Complete the testing and documentation for the HVTable component - Wrap up any todos or address any concerns with the project
Celebrate 🎉	

About Me

Commitments

I have no other commitments for the upcoming summer and the GSOC timeline. After completing my seventh semester, I may have institute exams, but the same would be over in under ten days. I am well versed in academics and can ensure that the same would help my dedication and work schedule.

Plans after GSoC

I love OpenSource, and working with the Leaning Equality team has been one of my favorite experiences. The fact that the team took the time to actively discuss and deliberate on a random suggestion that I provided related to testing and then even decided to incorporate the same is special to me! I would love to continue to be an active community member here as a code contributor and a guide who can help other Open Source enthusiasts stumbling upon the Kolibri ecosystem.

Planning for the Development Period:

I would like to work with the mentors to decide weekly goals to align myself and work in short sprints to build the required functionality. The timeline for the project, as mentioned earlier, would help me guide the overall development.

I am a good communicator, so I want to stay in close sync with the mentors via texts, Slack, or meetings to clear all the blocking factors quickly and focus more on development. I am a night owl, so I plan to devote 150 minutes (2.5 hours) every night, six days a week, to the project for 12 weeks, totaling 180 hours of work. I would happily put in extra hours if the project demands the same.

Appendix

Editing Functionality

To provide editing support for the cells of the table, there were primarily two options for the user journey:

- The user can directly click the cell, which changes it to an input box that he can edit, and then click a button or Enter to save.
- Clicking the editing button opens a new modal, prepopulated with the cell's value, which the user can edit.

I feel that the former approach is better from the point of view of accessibility as it provides immediate access where the users can directly interact with the cell without additional steps, reducing interaction costs. It is also a more intuitive approach, as clicking directly on the cell is a typical interaction pattern (popularized by spreadsheets), making it familiar to users. It also integrates seamlessly with keyboard navigation and shortcuts as outlined in the requirements,

The most concerning con of this approach comes from the lack of clear separation of editing mode: Opening a modal provides a distinct visual separation from the rest of the content, helping users focus on the editing task, lacking in the former approach. However, we can improve it by ensuring that the suitable cell stays focused before and after the editing and with the help of different input types for different kinds of data.

Sorting UX

There are three kinds of sorting behaviors possible. From a UX perspective, what happens when a user clicks repeatedly on the same header? Does it:

- Stay sorted in a single fixed order (Unusual)
- Toggle between ascending and descending order (Most common and the approach I plan on implementing)
- Cycle between ascending, descending, and some original (unsorted) order (applies to resources, but perhaps is not as widely used)

Rationale Behind Using Separate Components for Different Types of Tables (VS One Component to Service Them All)

As per my exploration, if we are to bundle all the functionalities of horizontal tables, vertical tables, as well as bidirectional tables into a single component, the same would complicate not only the implementation of the component but also the exported API, making it more difficult for the users to use.

Let us consider a simple example where the developer wants to create a table with a horizontal header in which some columns are editable and sortable. The final exported API may look something like:

```
<KTable
  type="horizontal"
  :editableColumns=[1,2,5],
  :sortableColumns=[1]
/>

<HTable
  :editable=[1,2,5],
  :sortable=[1]
/>
```

We can see a very subtle difference between the usage: The developer needs to supply additional information using a singular component. The same can be in the form of the additional prop “type” or can be the necessary additional quantification that the editable and sortable elements should be “columns” and not “rows.” On the other hand, using “HTable” and “VTable” like components conveys the intentions and reduces the developer's need to think about while using the component.

This point also highlights the unnecessary complexity of implementation that we would have to deal with. In the “KTable” example, we would need additional validator functions to ensure that the developer does not pass the “editableRows” or “editableColumns” props by mistake when the “type” is set to “horizontal.” No such requirements exist for the narrow-scoped “HTable” and “VTable” components, as these props simply do not exist.

Let us consider another simple example to drive the point of complexity further. In the case of HTable, the definition of the header row can be as simple as:

```
<!-- HTable -->
<tr>
  <td v-for="header in headers" :key="header" scope="col">
    {{ header }}
  </td>
</tr>
```

But if we use a single component for them, then each cell in the data can be either a header or a data cell, have an associated scope or not, and we would be required to handle the same via functions and conditionals unnecessary. Though we can't avoid this complexity altogether (as it would be needed in the bidirectional table), it makes the most sense to only opt into it when it is crucial. Otherwise, we should keep the code as simple as possible (so that it's easier to maintain and understand).

The only possible downside to this approach is some amount of code duplication between the components, but even the same can be easily avoided by opting into the right level of helper internal components like "KCell," "KRow," "KColumn," etc. which can have the generalized functionalities, and leave the component-specific behavior to their parents. These components can implement the primary sorting and editing logic, while the parent components can be responsible for allowing them to be editable/selectable/sortable via props.